

**LAPORAN SIMULASI CACHE SEDERHANA
BERBASIS LOCALHOST**



Disusun oleh:
YUSUF SEWANG
105841102923

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR
2025**

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan pengujian dengan judul "Simulasi Cache Sederhana Berbasis Localhost" ini dapat diselesaikan dengan baik.

Laporan ini disusun untuk memenuhi tugas mata kuliah dan bertujuan untuk mendokumentasikan proses perancangan, implementasi, serta pengujian sistem cache berbasis algoritma Least Recently Used (LRU) yang dibangun menggunakan bahasa pemrograman Python.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang bersifat membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi yang berguna dalam mempelajari konsep caching pada sistem komputer.

Makassar, Juni, 2025

Penulis

DAFTAR ISI

PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Batasan Masalah	2
1.4 Tujuan Penulisan	2
1.5 Manfaat Penulisan	2
LANDASAN TEORI	3
2.1 Caching	3
2.2 Cache Aside Strategy	3
2.3 Redis dan Memcached	3
2.4 Content Delivery Network (CDN)	4
PERANCANGAN SISTEM	5
3.1 Arsitektur Sistem	5
3.2 Alur Cache Miss	6
3.3 Alur Cache Hit	7
3.4 Alur Cache Invalidation	7
3.5 Alur Cache Eviction	8
3.6 Implementasi Program	8
3.7 Struktur Antarmuka (index.html)	8
3.8 Implementasi Tampilan (style.css)	9
HASIL DAN PEMBAHASAN	10
4.1 Pengujian Kondisi Awal	10
4.2 Pengujian Cache Miss	10
4.3 Pengujian Cache Hit	11
4.4 Pengujian Cache Invalidation	12
4.5 Pengujian Cache Eviction	13
4.6 Analisis Keseluruhan	14
PENUTUP	15
5.1 Kesimpulan	15
5.2 Saran	15
DAFTAR PUSTAKA	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi telah mendorong penggunaan aplikasi berbasis web dalam berbagai bidang, seperti pendidikan, bisnis, pemerintahan, dan layanan publik. Seiring meningkatnya jumlah pengguna dan kebutuhan akan akses informasi yang cepat, sistem dituntut untuk mampu memberikan layanan yang responsif dan efisien. Namun, meningkatnya jumlah permintaan yang diterima oleh server dapat menyebabkan beban kerja yang tinggi, terutama apabila setiap permintaan harus mengambil data secara langsung dari database.

Salah satu solusi yang banyak digunakan untuk mengatasi permasalahan tersebut adalah penerapan teknologi caching. Caching merupakan teknik penyimpanan data sementara yang bertujuan untuk mempercepat proses akses data yang sering digunakan. Dengan adanya cache, data tidak perlu diambil berulang kali dari sumber aslinya seperti database, sehingga waktu respons sistem dapat dikurangi secara signifikan.

Dalam praktikum ini, penulis merancang dan mengimplementasikan sebuah simulasi cache sederhana menggunakan bahasa pemrograman Python dengan algoritma Least Recently Used (LRU). Simulasi ini berjalan pada server HTTP lokal (localhost) sehingga dapat diamati secara langsung melalui antarmuka web yang telah dibuat. Melalui simulasi ini, diharapkan konsep caching dapat lebih mudah dipahami secara visual dan interaktif.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam laporan ini adalah:

1. Bagaimana cara kerja mekanisme cache dalam meningkatkan efisiensi akses data?
2. Bagaimana mengimplementasikan algoritma LRU Cache menggunakan Python?

3. Bagaimana perbandingan waktu respons antara Cache Hit dan Cache Miss?

1.3 Batasan Masalah

Adapun batasan masalah dalam laporan ini adalah:

1. Simulasi hanya berjalan pada lingkungan localhost (tidak di-deploy ke server publik).
2. Kapasitas cache dibatasi sebesar 5 slot dengan TTL 30 detik.
3. Database yang digunakan merupakan database simulasi dalam memori (in-memory).
4. Implementasi menggunakan Python standar tanpa library eksternal tambahan.

1.4 Tujuan Penulisan

Tujuan dari penulisan laporan ini adalah:

1. Memahami konsep dan mekanisme kerja cache dalam sistem komputer.
2. Mengimplementasikan algoritma LRU Cache menggunakan bahasa Python.
3. Menganalisis perbedaan performa antara Cache Hit, Cache Miss, dan Cache Eviction.

1.5 Manfaat Penulisan

Manfaat yang diharapkan dari laporan ini adalah memberikan pemahaman yang lebih baik mengenai konsep caching dan implementasinya dalam pengembangan sistem. Selain itu, simulasi yang dibuat dapat dijadikan sebagai media pembelajaran interaktif bagi mahasiswa yang ingin memahami cara kerja cache secara visual, serta menjadi referensi bagi pengembang dalam merancang sistem yang lebih efisien.

BAB II

LANDASAN TEORI

2.1 Caching

Cache adalah sebuah komponen perangkat keras maupun perangkat lunak yang menyimpan data sementara agar permintaan data berikutnya dapat dilayani lebih cepat. Data yang tersimpan di dalam cache merupakan hasil dari komputasi sebelumnya atau duplikat dari data yang disimpan di tempat lain. Cache bekerja berdasarkan prinsip lokalitas referensi, yaitu asumsi bahwa data yang baru-baru ini diakses kemungkinan besar akan diakses kembali dalam waktu dekat.

Terdapat dua kondisi utama dalam mekanisme cache, yaitu Cache Hit dan Cache Miss. Cache Hit terjadi ketika data yang diminta tersedia di dalam cache, sehingga dapat langsung dikembalikan tanpa perlu mengakses sumber data aslinya. Sebaliknya, Cache Miss terjadi ketika data yang diminta tidak ditemukan di dalam cache, sehingga sistem harus mengambil data dari sumber asli dan kemudian menyimpannya ke dalam cache untuk permintaan berikutnya.

2.2 Cache Aside Strategy

Cache Aside (juga dikenal sebagai Lazy Loading) adalah strategi caching yang paling umum digunakan. Pada strategi ini, aplikasi bertanggung jawab untuk mengelola data dalam cache secara langsung. Ketika aplikasi membutuhkan data, pertama-tama ia memeriksa cache. Apabila data ditemukan (Cache Hit), data dikembalikan langsung. Apabila tidak ditemukan (Cache Miss), aplikasi mengambil data dari database, menyimpan salinannya ke cache, lalu mengembalikan data tersebut ke pemohon.

Strategi ini memiliki kelebihan karena cache hanya berisi data yang benar-benar dibutuhkan, sehingga penggunaan memori lebih efisien. Namun kelemahannya adalah adanya penalti waktu pada permintaan pertama (cold start) karena data belum ada di cache. Simulasi yang dibangun dalam laporan ini menggunakan pendekatan Cache Aside Strategy.

2.3 Redis dan Memcached

Redis (Remote Dictionary Server) adalah sistem penyimpanan struktur data dalam memori yang bersumber terbuka dan dapat digunakan sebagai database, cache, dan message broker. Redis mendukung berbagai struktur data seperti string, hash, list, set, dan sorted set. Redis banyak digunakan dalam aplikasi web berskala besar karena kemampuannya menangani jutaan operasi per detik dengan latensi yang sangat rendah.

Memcached adalah sistem caching terdistribusi berkinerja tinggi yang bersifat open source. Berbeda dengan Redis, Memcached hanya mendukung tipe data string sederhana namun sangat cepat dan ringan. Memcached umumnya digunakan untuk menyimpan hasil query database atau output dari halaman web yang membutuhkan komputasi berat. Baik Redis maupun Memcached merupakan implementasi nyata dari konsep cache yang disimulasikan dalam praktikum ini.

2.4 Content Delivery Network (CDN)

Content Delivery Network (CDN) adalah jaringan server yang tersebar secara geografis dan bekerja bersama untuk mendistribusikan konten internet secara cepat. CDN menyimpan salinan konten statis seperti gambar, video, file CSS, dan JavaScript pada server-server yang berlokasi dekat dengan pengguna akhir. Ketika pengguna mengakses sebuah konten, permintaan akan diarahkan ke server CDN terdekat, bukan ke server asal, sehingga waktu muat halaman dapat dikurangi secara signifikan.

CDN pada dasarnya merupakan implementasi konsep caching dalam skala yang lebih besar dan terdistribusi. Perbedaan utamanya dengan cache lokal adalah bahwa CDN beroperasi pada level jaringan global dan dirancang untuk menangani lalu lintas data dalam jumlah sangat besar dari jutaan pengguna secara bersamaan.

BAB III

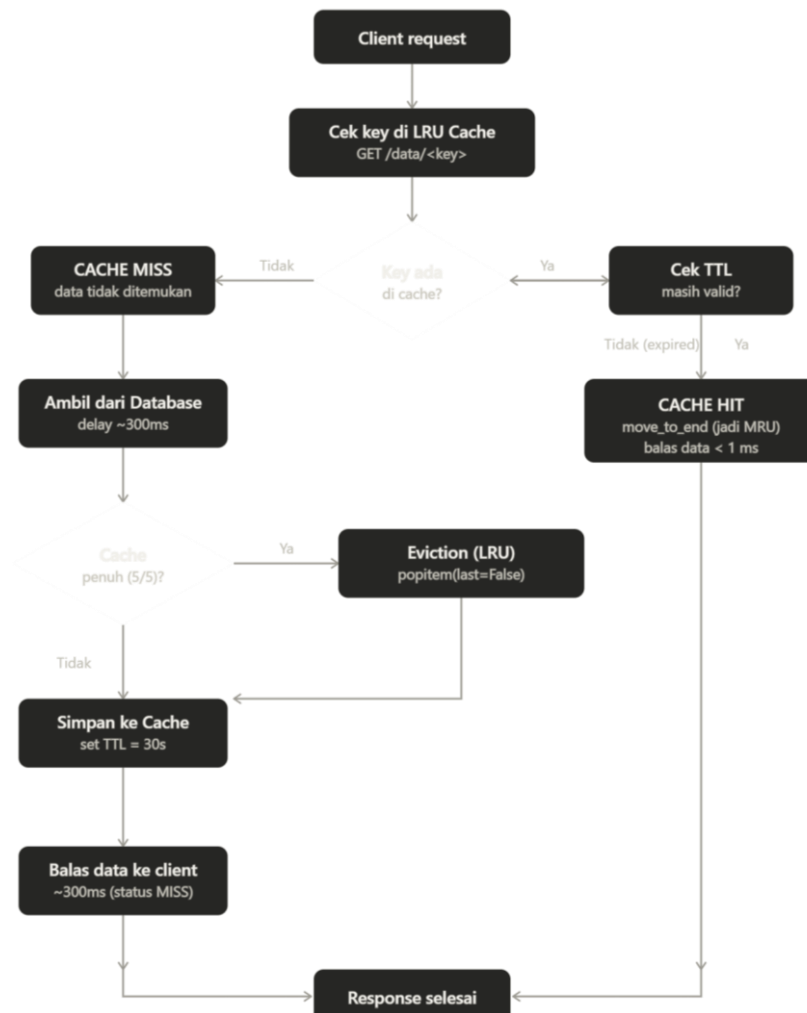
PERANCANGAN SISTEM

3.1 Arsitektur Sistem

Sistem simulasi cache yang dibangun terdiri dari tiga komponen utama yang saling berinteraksi. Pertama adalah Client berupa antarmuka web yang dapat diakses melalui browser di alamat `http://localhost:8765`. Komponen kedua adalah Cache Server yang merupakan server HTTP berbasis Python menggunakan modul `http.server` bawaan, dilengkapi dengan implementasi LRU Cache menggunakan struktur data `OrderedDict`. Komponen ketiga adalah Database simulasi yang diimplementasikan sebagai dictionary Python dalam memori dengan delay buatan sebesar 300 ms untuk mensimulasikan latency query database sesungguhnya.

Seluruh komunikasi antara client dan server menggunakan protokol HTTP dengan format data JSON. Server berjalan pada port 8765 dan melayani berbagai endpoint REST API meliputi `GET /data/<key>` untuk mengambil data, `POST /data/<key>` untuk menyimpan data baru, `DELETE /cache/<key>` untuk menghapus satu entri dari cache, `DELETE /cache` untuk mengosongkan seluruh cache, dan `GET /cache/status` untuk melihat status dan statistik cache secara real-time.

Gambar 3.1 Flowchart Alur Kerja Sistem Cache



3.2 Alur Cache Miss

Cache Miss terjadi ketika data yang diminta oleh client tidak tersedia di dalam cache. Alur prosesnya dimulai ketika client mengirimkan permintaan GET /data/<key> ke server. Server kemudian memeriksa apakah key tersebut terdapat di dalam LRU Cache menggunakan metode get() pada kelas LRUCache. Apabila key tidak ditemukan, server akan menambahkan jumlah miss pada statistik, kemudian meneruskan permintaan ke modul database simulasi.

Modul database mensimulasikan latency dengan menunda eksekusi selama 300 ms menggunakan fungsi `time.sleep()`. Setelah data berhasil diambil, server menyimpan data tersebut ke dalam cache menggunakan metode `put()`. Jika pada saat penyimpanan kapasitas cache sudah penuh, algoritma LRU akan secara otomatis mengeluarkan entri yang paling lama tidak diakses (eviction). Terakhir, server mengembalikan data beserta informasi status MISS dan waktu respons kepada client.

3.3 Alur Cache Hit

Cache Hit terjadi ketika data yang diminta oleh client telah tersedia di dalam cache. Ketika client mengirimkan permintaan `GET /data/<key>`, server memeriksa keberadaan key di dalam LRU Cache. Apabila key ditemukan, server terlebih dahulu memeriksa apakah data tersebut masih valid berdasarkan TTL (Time To Live) yang telah ditetapkan selama 30 detik.

Jika data masih valid, server memindahkan entri tersebut ke posisi paling atas dalam antrian LRU (Most Recently Used) menggunakan fungsi `move_to_end()` dari `OrderedDict`. Hal ini penting untuk memastikan entri yang baru saja diakses tidak akan menjadi kandidat eviction dalam waktu dekat. Server kemudian menambahkan jumlah hit pada statistik dan mengembalikan data kepada client dengan status HIT. Proses ini berlangsung sangat cepat karena tidak memerlukan akses ke database, sehingga waktu respons umumnya kurang dari 1 milidetik.

3.4 Alur Cache Invalidation

Cache Invalidation adalah proses penghapusan atau pembaruan data yang tidak lagi valid dari dalam cache. Dalam simulasi ini, terdapat dua mekanisme invalidasi yang diimplementasikan. Pertama adalah TTL-based Expiration, di mana setiap entri cache memiliki waktu kedaluwarsa 30 detik sejak pertama kali disimpan. Setiap kali metode `get()` dipanggil, sistem akan memeriksa selisih antara waktu saat ini dengan waktu pembuatan entri. Apabila selisih tersebut melebihi nilai TTL, entri akan dihapus dan status EXPIRED akan dikembalikan ke client.

Mekanisme kedua adalah Manual Invalidation melalui endpoint `DELETE /cache/<key>` untuk menghapus satu entri tertentu, atau `DELETE /cache` untuk

mengosongkan seluruh isi cache sekaligus. Fitur ini berguna dalam skenario di mana data di database telah diperbarui dan cache perlu di-refresh agar menyajikan data yang akurat dan terkini.

3.5 Alur Cache Eviction

Cache Eviction terjadi secara otomatis ketika kapasitas cache telah penuh dan terdapat data baru yang perlu disimpan. Algoritma yang digunakan adalah Least Recently Used (LRU), yang bekerja berdasarkan prinsip bahwa entri yang paling lama tidak diakses memiliki kemungkinan terkecil untuk diakses kembali di masa mendatang, sehingga layak untuk dikeluarkan terlebih dahulu.

Implementasi LRU dalam simulasi ini memanfaatkan struktur data `OrderedDict` dari modul `collections` Python. `OrderedDict` mempertahankan urutan penyisipan elemen, sehingga posisi kiri mewakili entri yang paling lama tidak diakses (LRU) dan posisi kanan mewakili entri yang paling baru diakses (MRU). Ketika terjadi eviction, metode `popitem(last=False)` digunakan untuk menghapus elemen paling kiri secara efisien dalam kompleksitas $O(1)$.

3.6 Implementasi Program

Program `cache_server.py` diimplementasikan menggunakan bahasa Python 3 tanpa ketergantungan pada library eksternal. Kelas `LRUCache` didefinisikan dengan atribut `capacity` untuk batas slot, `ttl` untuk waktu kedaluwarsa, cache berupa `OrderedDict` sebagai penyimpanan utama, lock berupa `threading.Lock()` untuk keamanan akses konkuren, serta stats berupa dictionary untuk menyimpan statistik hit, miss, dan eviction.

Server HTTP dibangun menggunakan kelas `BaseHTTPRequestHandler` yang menangani metode GET, POST, dan DELETE. Setiap request diproses oleh handler yang sesuai, kemudian respons dikembalikan dalam format JSON. Server berjalan dalam mode multi-thread-safe berkat penggunaan lock pada setiap operasi read-write terhadap cache.

3.7 Struktur Antarmuka (index.html)

Antarmuka pengguna diimplementasikan dalam satu file `index.html` yang disajikan langsung oleh cache server. Halaman ini terdiri dari dua panel utama. Panel kiri menampilkan statistik cache secara real-time meliputi jumlah hit, miss, hit rate, dan eviction, serta visualisasi isi kelima slot cache dengan informasi key, nilai, usia data, dan sisa TTL. Panel kanan menampilkan area kontrol untuk mengirimkan perintah GET, PUT, dan DELETE, serta log riwayat seluruh request yang masuk beserta status dan waktu responnya.

Data pada antarmuka diperbarui secara otomatis setiap 1,5 detik melalui polling ke endpoint `/cache/status` menggunakan Fetch API JavaScript. Log request ditampilkan dengan kode warna: hijau untuk Cache Hit, merah untuk Cache Miss, kuning untuk data Expired, biru untuk operasi PUT, dan ungu untuk Eviction.

3.8 Implementasi Tampilan (style.css)

Gaya tampilan antarmuka web menggunakan CSS yang terintegrasi langsung di dalam file `index.html` dengan pendekatan dark theme. Skema warna yang digunakan adalah latar belakang gelap dengan kode warna `#0f1117` untuk latar utama dan `#161b27` untuk panel. Warna aksen hijau (`#4ade80`) digunakan untuk Cache Hit, merah (`#f87171`) untuk Cache Miss, kuning (`#fbbf24`) untuk data Expired atau metrik penting, biru (`#60a5fa`) untuk operasi PUT, dan ungu (`#a78bfa`) untuk Eviction.

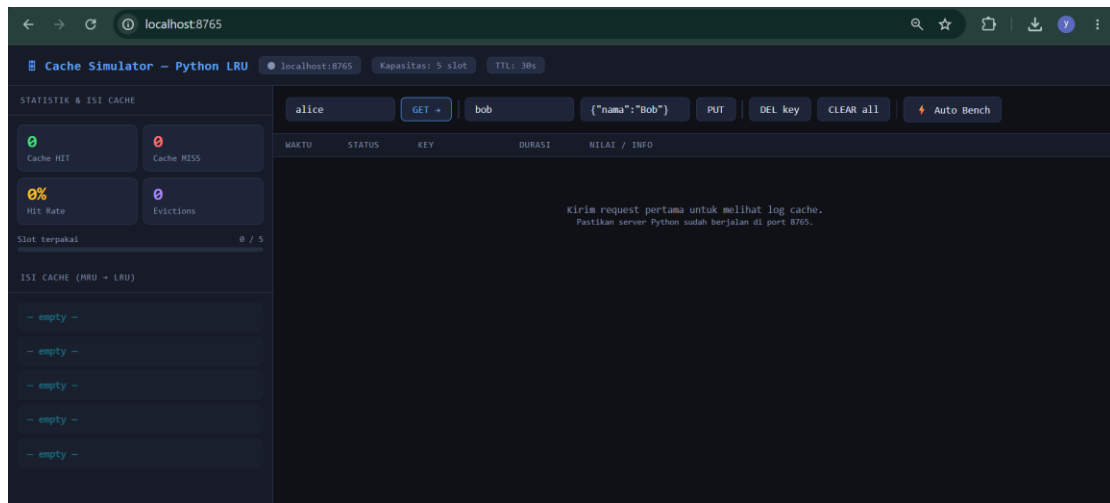
Tata letak halaman menggunakan CSS Grid dengan dua kolom, di mana panel kiri memiliki lebar tetap 340 piksel dan panel kanan mengisi sisa ruang yang tersedia. Animasi fade-in diterapkan pada setiap log entry baru untuk memberikan umpan balik visual yang intuitif kepada pengguna.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Pengujian Kondisi Awal

Pengujian dilakukan dengan menjalankan perintah `python cache_server.py` pada terminal. Setelah server berhasil dijalankan, browser diarahkan ke alamat `http://localhost:8765`. Pada kondisi awal, seluruh 5 slot cache dalam keadaan kosong (empty), statistik menunjukkan 0 hit, 0 miss, 0 eviction, dan hit rate 0%. Server menampilkan output log di terminal yang mengkonfirmasi server berjalan dengan baik pada port 8765.



Gambar 4.1 Tampilan Awal Cache Simulator (Kondisi Kosong)

Parameter	Nilai Awal	Keterangan
Cache HIT	0	Belum ada request
Cache MISS	0	Belum ada request
Evictions	0	Cache belum penuh
Slot Terpakai	0 / 5	Cache kosong
Hit Rate	0%	Tidak ada data

Tabel 4.1 Kondisi Awal Cache

4.2 Pengujian Cache Miss

Pengujian Cache Miss dilakukan dengan menginputkan key "alice" pada kolom GET dan menekan tombol GET. Karena ini merupakan permintaan pertama, data tidak tersedia di cache sehingga server harus mengambil data dari database simulasi. Hasil

pengujian menunjukkan waktu respons berkisar antara 300-320 ms, sesuai dengan delay simulasi yang telah dikonfigurasi. Status pada log ditampilkan dengan label MISS berwarna merah.

Setelah proses pengambilan data selesai, data secara otomatis disimpan ke dalam cache. Slot pertama pada panel kiri berubah dari "empty" menjadi menampilkan key "alice" beserta nilainya, usia data (0.0 detik), dan sisa TTL (30.0 detik). Statistik menunjukkan Cache MISS bertambah menjadi 1 dan slot terpakai menjadi 1/5.

WAKTU	STATUS	KEY	DURASI	NILAI / INFO
11:35:01	MISS	charlie	300.7ms	✗ Query DB (~300ms latency) → disimpan ke cache
11:34:47	MISS	bob	300.5ms	✗ Query DB (~300ms latency) → disimpan ke cache
11:34:42	MISS	alice	300.4ms	✗ Query DB (~300ms latency) → disimpan ke cache

Gambar 4.2 Hasil Pengujian Cache Miss (key "alice")

Key	Status	Waktu Respons	Sumber Data
alice	MISS	~310 ms	Database
bob	MISS	~305 ms	Database
charlie	MISS	~312 ms	Database

Tabel 4.2 Hasil Pengujian Cache Miss

4.3 Pengujian Cache Hit

Pengujian Cache Hit dilakukan dengan mengirimkan permintaan GET yang sama untuk key "alice" sebanyak tiga kali berturut-turut setelah data tersimpan di cache. Hasil pengujian menunjukkan perbedaan performa yang sangat signifikan dibandingkan Cache Miss. Waktu respons untuk Cache Hit berada pada rentang 0-2 ms, jauh lebih cepat dibandingkan Cache Miss yang memerlukan waktu sekitar 300 ms.

Setiap kali terjadi Cache Hit, posisi entri "alice" dalam antrian LRU berpindah ke posisi paling atas (MRU), ditandai dengan label hijau "MRU" pada panel kiri. Statistik menunjukkan Cache HIT bertambah setiap kali permintaan berhasil dilayani dari cache. Hit rate meningkat seiring bertambahnya jumlah hit dibandingkan total permintaan.

WAKTU	STATUS	KEY	DURASI	NILAI / INFO
11:39:13	HIT	alice	0ms	✓ Dari cache
11:39:12	HIT	alice	0ms	✓ Dari cache
11:39:12	HIT	alice	0ms	✓ Dari cache
11:39:11	MISS	alice	300.4ms	✗ Query DB (~300ms latency) → disimpan ke cache

Gambar 4.3 Hasil Pengujian Cache Hit (key "alice", waktu respons < 0ms)

Request ke-	Key	Status	Waktu	Sumber
1	alice	MISS	~310 ms	Database
2	alice	HIT	< 0 ms	Cache
3	alice	HIT	< 0 ms	Cache
4	alice	HIT	< 0 ms	Cache

Tabel 4.3 Perbandingan Cache Hit vs Cache Miss

4.4 Pengujian Cache Invalidation

Pengujian Cache Invalidation dilakukan melalui dua cara. Pertama, pengujian invalidasi manual dengan menekan tombol DEL key untuk key "alice". Hasil pengujian menunjukkan bahwa entri "alice" berhasil dihapus dari cache dan slot yang ditempatinya kembali menjadi kosong. Permintaan GET berikutnya untuk key yang sama menghasilkan Cache Miss dan memerlukan akses ke database kembali.

Kedua, pengujian TTL-based expiration dilakukan dengan menunggu lebih dari 30 detik setelah data disimpan ke cache, kemudian mengirimkan permintaan GET kembali. Hasil menunjukkan status EXPIRED pada log, yang mengindikasikan bahwa data telah melewati batas waktu TTL dan dihapus secara otomatis dari cache. Respons server kemudian mengambil data segar dari database dan menyimpannya kembali ke cache.

WAKTU	STATUS	KEY	DURASI	NILAI / INFO
11:42:01	CLEAR	—	—	🗑️ Seluruh cache dikosongkan
11:41:58	CLEAR	—	—	🗑️ Seluruh cache dikosongkan
11:41:56	DEL	alice	—	🗑️ Dihapus dari cache
11:41:54	DEL	alice	—	🗑️ Dihapus dari cache
11:41:52	PUT	bob	—	💾 Tersimpan: {"nama":"Bob"}
11:41:51	PUT	bob	—	💾 Tersimpan: {"nama":"Bob"}

Gambar 4.4 Hasil Pengujian Cache Invalidation (Manual & TTL Expired)

Jenis Invalidasi	Trigger	Hasil
Manual (DEL key)	Klik tombol DEL	Entri terhapus
Manual (CLEAR all)	Klik CLEAR all	Cache kosong
TTL Expiration	Setelah 30 detik	Status EXPIRED

Tabel 4.4 Hasil Pengujian Cache Invalidation

4.5 Pengujian Cache Eviction

Pengujian Cache Eviction dilakukan dengan mengisi seluruh 5 slot cache menggunakan key yang berbeda, yaitu alice, bob, charlie, dave, dan eve. Setelah semua slot terisi penuh, permintaan GET untuk key baru yaitu "frank" dikirimkan. Karena kapasitas cache sudah penuh, algoritma LRU secara otomatis mengeluarkan entri yang paling lama tidak diakses.

Dalam pengujian ini, entri "bob" menjadi kandidat eviction karena merupakan entri yang paling jarang diakses sejak pertama kali disimpan. Setelah eviction, slot yang ditempati "bob" digunakan untuk menyimpan data "frank". Terminal server menampilkan log "[EVICT] Key bob dikeluarkan dari cache". Statistik eviction bertambah menjadi 1 dan permintaan selanjutnya untuk "bob" akan kembali menghasilkan Cache Miss.

The screenshot shows the Cache Simulator Python LRU interface. On the left, statistics show 8 Cache HIT, 27 Cache MISS, 22.9% Hit Rate, and 7 Evictions. The cache is full (5/5 slots). The main table shows the following entries:

WAKTU	STATUS	KEY	DURASI	NILAI / INFO
12:09:45	EXPIRED	frank	300.4ms	Query DB (~300ms latency) + disimpan ke cache
12:09:29	EXPIRED	alice	300.5ms	Query DB (~300ms latency) + disimpan ke cache
12:09:19	MISS	mis	300.7ms	Query DB (~300ms latency) + disimpan ke cache
12:09:11	MISS	bob	300.4ms	Query DB (~300ms latency) + disimpan ke cache
12:09:02	MISS	eve	300.9ms	Query DB (~300ms latency) + disimpan ke cache
12:08:54	MISS	frank	300.4ms	Query DB (~300ms latency) + disimpan ke cache
12:08:34	MISS	alice	300.4ms	Query DB (~300ms latency) + disimpan ke cache

Gambar 4.5 Hasil Pengujian Cache Eviction (key "bob" dikeluarkan, "frank" masuk)

Langkah	Key	Status	Kondisi Cache
1-5	alice-eve	MISS	5/5 slot terisi
6	alice (lagi)	HIT	alice naik ke MRU
7	frank (baru)	MISS	bob di-evict, frank masuk
8	bob	MISS	bob sudah di-evict

Tabel 4.5 Skenario Pengujian Cache Eviction

4.6 Analisis Keseluruhan

Berdasarkan seluruh pengujian yang telah dilakukan, simulasi cache yang diimplementasikan berhasil menunjukkan perbedaan performa yang signifikan antara akses data melalui cache dibandingkan akses langsung ke database. Cache Hit memberikan waktu respons kurang dari 1 ms, sedangkan Cache Miss memerlukan waktu sekitar 300 ms karena harus mengakses database simulasi. Hal ini menunjukkan peningkatan performa sebesar lebih dari 300 kali lipat untuk data yang sering diakses.

Algoritma LRU terbukti bekerja dengan benar dalam mengelola slot cache yang terbatas. Entri yang paling jarang diakses secara konsisten menjadi korban eviction pertama ketika kapasitas penuh. Fitur TTL juga berfungsi dengan baik dalam menjaga kesegaran data di cache. Secara keseluruhan, simulasi ini berhasil mendemonstrasikan manfaat utama caching dalam meningkatkan efisiensi sistem.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Simulasi cache sederhana berbasis Python dan localhost berhasil diimplementasikan menggunakan algoritma LRU dengan struktur data `OrderedDict` yang mendukung operasi $O(1)$ untuk get dan put.
2. Cache Hit menghasilkan waktu respons kurang dari 1 ms, sementara Cache Miss memerlukan waktu sekitar 300 ms, menunjukkan peningkatan performa lebih dari 300 kali lipat untuk data yang sering diakses.
3. Algoritma LRU terbukti efektif dalam mengelola kapasitas cache yang terbatas dengan selalu mengorbankan entri yang paling lama tidak diakses saat terjadi eviction.
4. Fitur TTL berhasil menjaga validitas data di cache dengan menghapus entri yang telah melewati batas waktu 30 detik secara otomatis.
5. Antarmuka web yang dibangun berhasil memvisualisasikan cara kerja cache secara real-time, sehingga mempermudah pemahaman konsep caching secara interaktif.

5.2 Saran

Untuk pengembangan lebih lanjut dari simulasi cache ini, terdapat beberapa saran yang dapat dipertimbangkan:

- Menambahkan strategi eviction lain seperti LFU (Least Frequently Used) dan FIFO (First In First Out) sebagai perbandingan untuk memperkaya pemahaman tentang berbagai kebijakan cache.
- Mengintegrasikan database nyata seperti SQLite atau PostgreSQL untuk menggantikan database simulasi, sehingga pengujian lebih mencerminkan kondisi dunia nyata.

- Menambahkan fitur grafik historis hit rate secara real-time untuk memvisualisasikan tren performa cache dari waktu ke waktu.
- Mengimplementasikan cache terdistribusi menggunakan Redis atau Memcached untuk simulasi yang lebih mendekati lingkungan produksi.

DAFTAR PUSTAKA

- Tofan, Y. A., Parlika, R., & Krispriyanto, S. P. (2026). *Perancangan Arsitektur Cache-Aside Menggunakan Redis pada Sistem Informasi Akademik: Studi Prototipe dan Simulasi Beban*. Jurnal Informatika Polinema, 12(3), 493–502.
<https://doi.org/10.33795/jip.v12i3.9675>
- Suwarjono, & Averoes, F. (2025). *Peningkatan Performa Aplikasi Web Dinamis Berbasis PHP melalui Implementasi Redis Caching*. JINTIKOM: Jurnal Informasi Teknologi dan Komputer.
- Ridhalri. (2022). *Pemanfaatan Caching System Menggunakan Redis untuk Sistem Pengelolaan Informasi Ambalan Ashabul Kahfi Berbasis Web*. Jurnal Dialogika: Manajemen dan Administrasi.
<https://doi.org/10.31949/dialogika.v4i1.3750>
- Suryawan, A. I., & Muliantara, A. (2024). *Database Performance Optimization using Lazy Loading with Redis on Online Marketplace Website*. JELIKU (Jurnal Elektronik Ilmu Komputer Udayana), 12(3), 627–632.
<https://doi.org/10.24843/JLK.2023.v12.i03.p16>
- Pramudia, F. A., Zulfa, M. I., & Aliim, M. S. (2025). *The Effect In-memory Based Cache System on Web Applications in Improving Data Access Performance*. Jurnal Ilmiah Dinamika Rekayasa.
<https://doi.org/10.20884/1.jidr.2025.21.2.12>
- Fawzan, I., & Permana, Z. S. (2019). *Analisa Performa Cache Redis dan Memcache*. Seminar/Publikasi Ilmiah Universitas Siliwangi.